



ಗೋಬಲ್ ಅಕಾಡೆಮಿ ಆಫ್ ಟೆಕ್ನಾಲಜಿ, ಬೆಂಗಳೂರು
GLOBAL ACADEMY OF TECHNOLOGY

An Autonomous Institute Affiliated to VTU-Belagavi, Approved by AICTE and Accredited by NAAC, "A" grade

Aditya Layout, Ideal Homes Township, RR Nagar, Bengaluru, Karnataka 560098

Department of Computer Science and Engineering

Accredited by NBA 2022 - 2025



+91 80 28603158(Ext - 300)



hodcse@gat.ac.in



www.gat.ac.in

LAB MANUAL

Cloud Computing Lab Manual (Integrated)

VI Semester

Course Code: 22CSE61

Version 1.0

w.e.f 10.03.2025

Editorial Committee

Cloud Computing Laboratory Faculty,

Dept. of CSE

Approved by

Dr. Kumaraswamy S

Professor & Head,

Department of CSE



ಗ್ಲೋಬಲ್ ಅಕಾಡೆಮಿ ಆಫ್ ಟೆಕ್ನಾಲಜಿ, ಬೆಂಗಳೂರು
GLOBAL ACADEMY OF TECHNOLOGY

An Autonomous Institute Affiliated to VTU-Belagavi, Approved by AICTE and Accredited by NAAC, "A" grade

Aditya Layout, Ideal Homes Township, RR Nagar, Bengaluru, Karnataka 560098

Department of Computer Science and Engineering

Accredited by NBA 2022 - 2025



+91 80 28603158(Ext - 300)



hodcse@gat.ac.in



www.gat.ac.in

LABORATORY CERTIFICATE

This is to certify that Mr./ Ms

bearing USN..... of Computer Science and Engineering

Department has satisfactorily completed the course of Cloud Computing Laboratory

(Integrated) prescribed by Global Academy of Technology (Autonomous Institute, under

VTU) for the Academic Year 2024-2025

Marks	
Maximum	Obtained
20	

Signature of the Faculty

Signature of the HOD

Date:

DOCUMENT LOG

Name of the document	Cloud Computing Laboratory (Integrated)
Syllabus Scheme	2022
Current version number and date	V1 / 10.03.2025
Subject code	22CSE61
Editorial Committee	Cloud Computing Laboratory Faculty
Approved by	HOD, Dept. of CSE
Lab Faculty	Prof. Haripriya C Prof. Thaseen Taj Prof. Archana M
Computer Programmer	Mr. Parashivamurthy C

Table of Contents

Sl.No.	Particulars.	Page No.
1	Vision and Mission of The Department	1
2	PEOs, PSOs, POs	2-3
3	Syllabus • Course Objectives • Course Outcomes • Conduction of Practical Examination • CO-PO-PSO Mapping	4-5
4	Scheme of Evaluation	6
5	Marks Distribution	6
6	Rubrics	7
7	PROGRAMS	
	Program 1 Install VirtualBox or VMware Workstation on a Windows 11 system, set up a virtual machine with a Linux or Windows operating system, install a C compiler, and execute simple C programs.	8
	Program 2 Find a procedure to transfer the files from One Virtual Machine to another Virtual Machine .	10
	Program 3 Deploy a Flask web application to Google Cloud using Google Cloud CLI as PaaS .	12
	Program 4 Build a Docker image for a simple Flask web application and push it to your Docker Hub repository.	14
	Program 5 Design and implement a multi-container using Docker Compose to deploy Apache Kafka and Zookeeper.	16
	Program 6 Pull a Docker image from Docker Hub and deploy your Flask app using Docker Swarm.	18
	Program 7 Find vulnerabilities, misconfigurations and secrets in files, Docker, DockerHub and Git Repo using Trivy .and generate a summary report in HTML format.	19
	Program 8 Simulate and Manage Amazon S3 buckets and EC2 using LocalStack locally. Perform operations using both the LocalStack Cloud Dashboard and AWS CLI.	22
	Program 9 Simulate and manage a Amazon DynamoDB Using AWS CLI with AWS LocalStack Cloud.	24
	Program 10 Design and Develop a custom Student Registration Form application using Salesforce as a SaaS.	27

Vision of the Department

To achieve academic excellence and strengthen the skills to meet emerging challenges of Computer Science and Engineering.

Mission of the Department

- M1:** To impart strong theoretical foundation in the field of Computer Science and Engineering accompanied with extensive practical skills.
- M2:** To inculcate research and innovation spirit through interaction with industry and carry out projects that address societal needs.
- M3:** Instill professional ethics and values with concern for environment.

Program Educational Objectives (PEOs) of Department

After the course completion, CSE graduates will be able to:

PEO1: Succeed in engineering/management positions with professional ethics.

PEO2: Engage in improving professional knowledge through certificate /post- graduate programs in engineering or management.

PEO3: Establish themselves as entrepreneurs and contribute to the Society.

Program Specific Outcomes (PSOs)

PSO1: Design, implement and test System Software and Application Software to meet the desired needs.

PSO2: Develop solutions in the area of Communication, Networks, database systems and computing systems.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

SYLLABUS

Subject Code	22CSE61	CIE Marks	50
Hours/Week (L: T: P)	0:0:2	SEE Marks	50
Credits	4	Examination Hours	03

Course Learning Objectives

CLO1	Illustrate the core concepts of cloud computing paradigm and reference models.
CLO2	Discuss system virtualization and outline its role in enabling the cloud computing system model.
CLO3	Understand cloud management and cloud security.
CLO4	Analyse the different applications of cloud computing

Lab Programs:

1. Install Virtual Box or VMware Workstation on a Windows 11 system, set up a virtual machine with a Linux or Windows operating system, install a C compiler and execute simple C programs.
2. Find a procedure to transfer the files from One Virtual Machine to another Virtual Machine.
3. Deploy a Flask web application to Google Cloud using Google Cloud CLI as PaaS.
4. Build a Docker image for a simple Flask web application and push it to your Docker Hub repository.
5. Design and implement a multi-container using Docker Compose to deploy Apache Kafka and Zookeeper.
6. Pull a Docker image from Docker Hub and deploy your Flask app using Docker Swarm.
7. Find vulnerabilities, misconfigurations and secrets in files, Docker, Docker Hub and Git Repo using Aqua Trivy . and generate a summary report in HTML format.
8. Simulate and Manage Amazon S3 buckets and EC2 using LocalStack locally. Perform operations using both the LocalStack Cloud Dashboard and AWS CLI.
9. Simulate and manage a Amazon DynamoDB Using AWS CLI with AWS LocalStack Cloud.
10. Design and Develop a custom Student Registration Form application using Salesforce as a SaaS.

Course Outcomes

Upon successful completion of this course, students are able to

CO1	Explain the concepts and terminologies of cloud computing
CO2	Understand the concepts of Virtualization
CO3	Identify and analyze the reference models for Cloud Computing
CO4	Examine the Cloud Environment & Cloud Security
CO5	Interpret the use of AWS, Azure and Google cloud platform to develop application

CO-PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	1	1	2							2	2	
CO2	3	2	1	1	2							2	2	
CO3	3	2	1	1	2							2	2	
CO4	3	2	1	1	2							2	2	
CO5	3	2	1	1	2							2	2	
AVG	3	2	1	1	2							2	2	

Scheme of Evaluation

(Integrated courses)

Semester End Examination (SEE):

SEE Question paper is to be set for 100 marks and the marks scored will be proportionately reduced to 50. There will be two full questions (with a maximum of four sub questions) from each module carrying 20 marks each. Students are required to answer any **five full questions** choosing at least **one full question from each module**.

The laboratory assessment would be restricted to only the CIE evaluation.

Continuous Internal Evaluation (CIE):

Continuous Internal Evaluation (CIE): Three Tests are to be conducted for **40 marks each**. The average of the three tests are taken for computation of CIE on a scale of 30, the CIE would also include laboratory evaluation for 20 marks. The laboratory marks of 20 would comprise of 10 marks for regular laboratory assessment to include lab record and observation. 10 marks would be exclusive for laboratory internal assessment test to be conducted at the end of the semester.

Table: Distribution of weightage for CIE & SEE of Integrated courses

	Component	Marks	Total Marks
CIE	Average of Three CIEs (Min Passing Marks:12)	30	50
	Laboratory (Min Passing Marks:8)	20	
SEE	Semester End Examination (Min Passing Mark:40)	100	50
Grand Total			100

LAB EVALUATION PROCESS

Sl.No	Parameter	Max. Marks (Evaluation)	Reduced to
1	Record & Observation	30 Marks per week	10
2	Final Lab Internal Assessment	100	10
Total			20

LAB RUBRICS

Rubrics for Evaluation of Record and Observation

Attribute	Max. Marks	Good	Satisfactory	Poor
		4-5	2-3	0-1
Write-up	05	Writes the program without errors.	Writes the program with few mistakes.	Unable to write the program.
	Max. Marks	10-20	5-9	0-4
Execution of program	20	• Debugs the program independently. • Executed the program for all possible inputs.	• Works with little help from faculty. • All possible input cases not covered.	• Unable to complete the execution of the program within the lab session.
	Max. Marks	3-5	1-2	0
Viva Voce	05	• Able to explain the logic of the program. • Answered all questions.	• Partially understood the logic of the program. • Answered few questions.	• Not understood the logic of the program. • Not answering any questions

Rubrics for Evaluation of Internal Test Part A

Attribute	Max Marks	Good	Satisfactory	Poor
		4-5	2-3	0-1
Write-up	10	• Completes source code. • No syntax and logical errors. • All possible inputs listed with expected output.	• Completes source code. • Syntax and logical errors exist. • All possible inputs listed with expected output.	• Incomplete source code. • Change of program.
	Max Marks	21-30	6-20	1-5
Execution	30	Able to debug the program and get the right set of outputs.	• Program, executed for only few input cases.	Not executed.
	Max Marks	3-5	1-2	0
Viva Voce	10	Answering all questions.	Answering few questions.	Not Able to answer basic questions.

Program 1.

Install VirtualBox or VMware Workstation on a Windows 11 system, set up a virtual machine with a Linux or Windows operating system, install a C compiler, and execute simple C programs.

Prerequisite Software

1. Installed Windows 11 OS
2. VMware Workstation
3. Ubuntu 18.04

Implementation Steps

Step1: Install VMware Workstation

Step2. Click on Create a New Virtual Machine

Step3: Select Types of Configuration and click Next

Step4: Select the Disc image of Ubuntu 18 and click Next

Step5: Enter the Personalize details to create a User.

Ex:

Full Name: Global

User name: gat

Password: root123

Conform: root123

and click Next.

Step6: Enter the Virtual Machine Name, Keep the default location setting as it is, then click **Next** to proceed.

Step7: Select the necessary hardware specifications (such as memory, processors, and click Next

EX:

Processor: 2 CPUs

RAM : 2 GB

Hard Disk: 20 GB

Step 8: Click Finish

Step 9: Install the GCC Compiler

```
sudo apt update
```

```
sudo apt install gcc
```

Step 10: Create a C Program

```
#include <stdio.h>
int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

Step 11: Compile the C Program

`./a.out`

Program 2.

Find a procedure to transfer the files from One Virtual Machine to another Virtual Machine.

Prerequisite Software

1. Installed Windows 11 OS
2. VMware Workstation
3. Ubuntu 18.04

Implementation Steps

Creation of Virtual Machine 1

Step1: Click on Create a New Virtual Machine

Step2: Select Types of Configuration and click Next

Step3: Select the Disc image of Ubuntu 18 and click Next

Step4: Enter the Personalize details to create a User.

Ex:

Full Name: Global1

User name: gat1

Password: root123

Conform: root123

and click Next.

Step5: Enter the Virtual Machine Name, Keep the default location setting as it is, then click **Next** to proceed.

Step6: Select the necessary hardware specifications (such as memory, processors, and click Next

EX:

Processor: 2 CPUs

RAM : 2 GB

Hard Disk: 20 GB

Step7: Click Finish

Step8: to know the IP address of the respective Virtual machines.

```
sudo apt install net-tools
```

```
Ifconfig
```

Ex:

The IP address of the Virtual Machine 1 is **192.168.90.128**

Step9 : Install and start the OpenSSH Server

```
sudo apt install openssh-server -y
```

Creation of Virtual Machine 2

Step1: Click on Create a New Virtual Machine

Step2: Select Types of Configuration and click Next

Step3: Select the Disc image of Ubuntu 18 and click Next

Step4: Enter the Personalize details to create a User.

Ex:

Full Name: Global2

User name: gat2

Password: root123

Conform: root123

and click Next.

Step5: Enter the Virtual Machine Name, Keep the default location setting as it is, then click **Next** to proceed.

Step6: Select the necessary hardware specifications (such as memory, processors, and click Next

EX:

Processor: 2 CPUs

RAM : 2 GB

Hard Disk: 20 GB

Step7: Click Finish

Step8: to know the IP address of the respective Virtual machines.

```
sudo apt install net-tools
```

```
Ifconfig
```

Ex:

The IP address of the Virtual Machine 1 is **192.168.90.129**

Step9: Install and start the OpenSSH Server

```
sudo apt install openssh-server -y
```

transfer the files from gat1 to another gat2

Step 1: Creating a file named p1.txt and p2.txt on **gat1**

Step 2. Transfer a file through **scp** and **rsync** from **gat1** to **gat2**

Ex1: scp p1.txt gat2@192.168.90.129:home/gat2

Ex2: rsync -avz p2.txt gat2@192.168.90.129:home/gat2

Step 3. Check if the files have been successfully transferred from **gat1** to **gat2**

Program 3.

Deploy a Flask web application to Google Cloud using Google Cloud CLI as PaaS.

Prerequisite Software

1. Google Cloud CLI installer.
2. Email ID
3. Python 3.9.6
- 4: Flask
5. Install the App Engine component

Implementation Steps

1. Create Google Cloud Project ID

URL : <https://console.cloud.google.com/terms>

Step 1: Enter Project Name and Click on Create

Step 2: Open Terminal, Type **gcloud init** to reinitialize

Step 3: Choices 1 to reinitialize

Step 4: select your Account

Step 5: Select Project ID

Step 6: Create a new folder in C Drive: WITH A NAME AS **flask-app**

Step 7: Create a file named app.yaml

```
runtime: python39
entrypoint: python main.py # Directly run Flask

handlers:
- url: /*
  script: auto
```


Step 8: Create a file named **main.py**

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

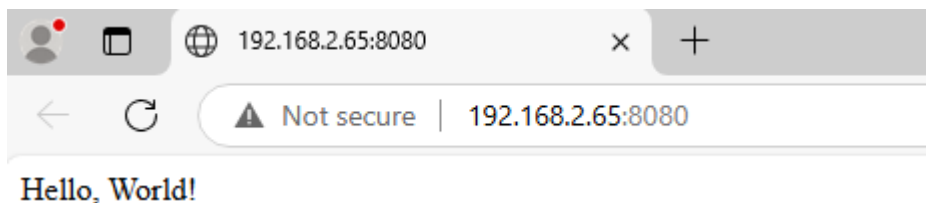
Step 9: Create a file named: requirements.txt

```
flask
```

Step 10: Run **the app locally - before deploying to Google Cloud**

```
python "C:\Users\Admin\AppData\Local\Google\Cloud SDK\google-cloud-sdk\bin\dev_appserver.py" app.yaml
```

Step 11: type on browser 192.168.2.65:8080



Step 12: gcloud app deploy - This CM is used Deploy the app to Google Cloud

Step 13: Select any region by pressing the relevant button

Step 14: gcloud app browse , this CMD is used to View the live app

Program 4.

Build a Docker image for a simple Flask web application and push it to your Docker Hub repository.

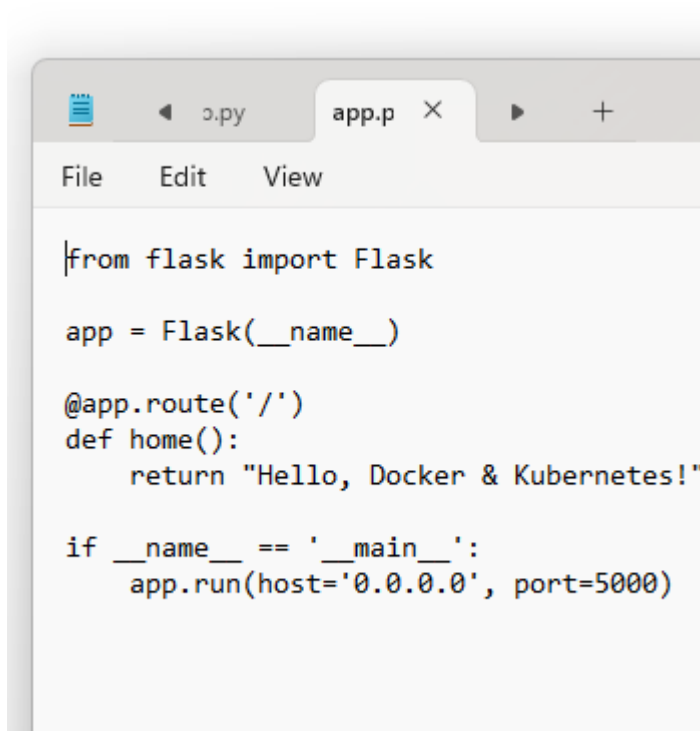
1. Prerequisite Software

1. Docker Desktop
2. Wsl
3. Docker Hub Account

2. Implementation Steps

Step 2.1: Create a new folder “flask-app” in C drive

Step 2.2: Create a simple python file inside flask-app “app.py” Under the Flask-app Folder
app.py

A screenshot of a code editor window with a tab labeled 'app.p'. The editor shows Python code for a Flask application. The code includes imports, app initialization, a home route, and a main block to run the app on port 5000.

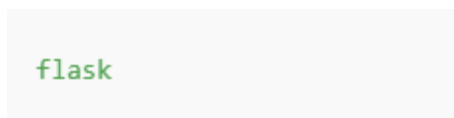
```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Docker & Kubernetes!"

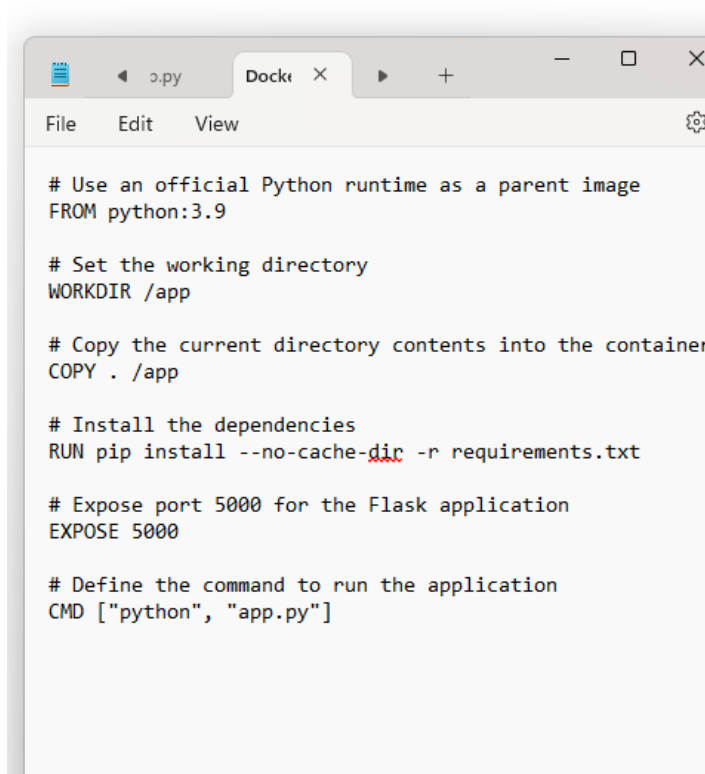
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 2.3: Create a new text file in “flask-app” has “requirement.txt”

A screenshot of a text file named 'requirement.txt' containing the word 'flask' in green text.

```
flask
```

Step 2.4: Inside “flask-app” Create a Dockerfile should be in file format.

A screenshot of a code editor window titled 'Dockerfile'. The editor shows a Dockerfile with the following content:

```
# Use an official Python runtime as a parent image
FROM python:3.9

# Set the working directory
WORKDIR /app

# Copy the current directory contents into the container
COPY . /app

# Install the dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose port 5000 for the Flask application
EXPOSE 5000

# Define the command to run the application
CMD ["python", "app.py"]
```

Step 2.5: Create Build a Docker image by Clicking Terminal inside the Desktop docker

`docker build -t flask-app .`

Step 2.6: To check the images

`docker images`

Step 2.7: Run a Docker image in container

`docker run -p 5000:5000 -d flask-app`

Step 2.8 :To check image is running

`docker ps -a`

Step 2.9: Type <http://localhost:5000> on your browser

Step 2.10: Tag the image before pushing to Docker Hub

Ex : `docker tag flask-app parashivamurthyc/flask-app:latest`

Step 2.11: to push image to the Docker Hub

Ex : `docker push parashivamurthyc/flask-app:latest`

Step 2.12: To verify The Image(parashivamurthyc/flask-app:latest) in Docker Hub

Program 5.

Design and implement a multi-container using Docker Compose to deploy Apache Kafka and Zookeeper.

1. Prerequisite Software

1.1 Docker Desktop

2. Implementation Steps

2.1. Create a Project Directory in C Drive, Name as **Kafka**

2.2 Create a file named `docker-compose.yml` in **Kafka** Folder

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.0.1
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:7.0.1
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

2.3 Run Docker Desktop

2.4 Change The Default Directory Path To kafka

2.5 to starts all services

`docker-compose up -d`

2.6 Open Two Windows PowerShell Terminals

in Terminal 1: Run → Kafka , to Produce Messages

2.7 to Run Kafka

```
docker exec -it kafka bash
```

2.8 to Create a topic

```
kafka-topics --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

2.9 Verify topic is created

```
kafka-topics --list --bootstrap-server localhost:9092
```

2.10 Produce message

```
kafka-console-producer --broker-list localhost:9092 --topic test-topic
```

2.11 Type your messages and hit Enter button

In Terminal 2: Run → Kafka , to Consume Message

2.12 Run Kafka

```
docker exec -it kafka bash
```

2.13. to Consume Message

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic test-topic --from-beginning
```

Program 6.

Pull a Docker image from Docker Hub and deploy your Flask app using Docker Swarm.

Prerequisites software

- Docker installed
- Docker Swarm initialized (`docker swarm init`) on **manager node**
- Docker Hub or a registry (e.g., `shwethapmurthy/flask-app2:latest`)
- A `docker-compose.yml` file prepared for the service

Implementation Steps

Step 1: Create a folder and a file

1.1. Create a Project Directory in C Drive, Name as **Docker-Swarm**

1.2. Create a file named `docker-compose.yml` inside the Docker-Swarm folder.

Step 2: Initialize / Enable Docker Swarm (on manager node)

```
docker swarm init
```

Step3: View All Nodes in the Cluster

```
docker node ls
```

Step 4: Deploy Stack to Swarm

```
docker stack deploy -c docker-compose.yml flaskstack
```

Step 5: Verify Swarm Services

```
docker stack services flaskstack
```

Step 6: Check service tasks (containers): - Docker Swarm creates **2 tasks (containers)**

```
Docker stack ps flaskstack
```

Step 7: Access Your Flask App

type **`http://localhost:5000`** on your browser

Step 8: stop and remove the entire stack of Docker Swarm

```
docker stack rm flaskstack
```

Step 9: Leave Swarm

```
docker swarm leave -force
```

Program 7.

Find vulnerabilities, misconfigurations and secrets in files, Docker Image using **Trivy** . and generate a summary report in HTML format.

1.Prerequisite Software

1.1 Aqua Trivy

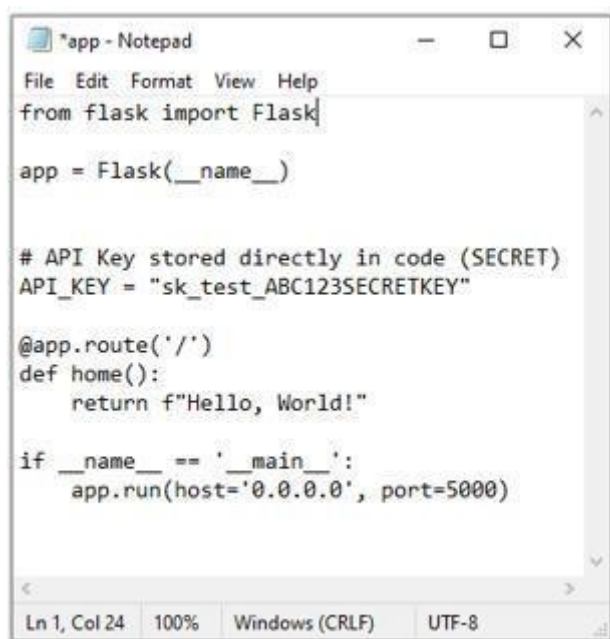
1.2 Docker Desktop

2.Implementation Steps

2.1 Create a directory named `trivy` in C drive. Within this directory, create the following

three files: `app.py`, `Dockerfile` , and `requirements.txt`."

2.2 `app.py` - for Secrets



```
*app - Notepad
File Edit Format View Help
from flask import Flask

app = Flask(__name__)

# API Key stored directly in code (SECRET)
API_KEY = "sk_test_ABC123SECRETKEY"

@app.route('/')
def home():
    return f"Hello, World!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

Ln 1, Col 24 100% Windows (CRLF) UTF-8
```

2.3 Dockerfile - for Misconfigurations



```
File Edit Format View Help
# Use an official Python runtime as a parent image
FROM python:3.9

# Set the working directory inside the container
WORKDIR /app

# Copy app files
COPY . /app

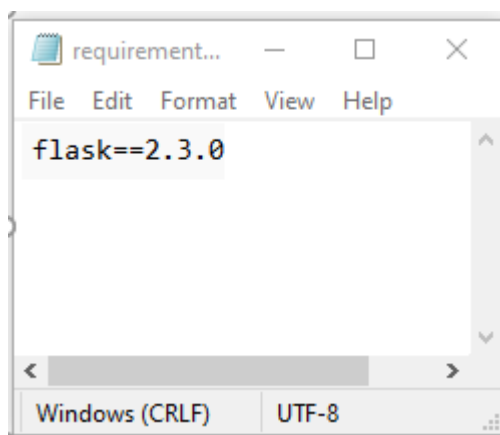
# Install the dependencies
RUN pip install -r requirements.txt

# Expose port 5000 for the Flask app
EXPOSE 5000

# Run the Flask app
CMD ["python", "app.py"]
```

Ln 7, Col 17 100% Windows (CRLF) UTF-8

2.4 requirements.txt - for Vulnerabilities



```
File Edit Format View Help
flask==2.3.0
```

Windows (CRLF) UTF-8

2.5 To generate a Summary Report in HTML format

First, download the original HTML template from Trivy:

🔗 <https://github.com/aquasecurity/trivy/blob/main/contrib/html.tpl>

Click **“Raw”** → Right-click → **“Save As”** → Save as `html.tpl` inside of **Trivy** Folder

Run in CLI Mode to generate the Report Summary in **Trivy** Folder

2.6 full scan (vulnerabilities, misconfigurations and secrets)

```
trivy fs --format template --template "@html.tpl" -o fullscan-report.html --security-checks secret,config,vuln .
```

2.7 Individual Scan for secret

```
trivy fs --format template --template "@html.tpl" -o secret-report.html --security-checks secret .
```

2.8 Individual Scan for misconfigurations

```
trivy fs --format template --template "@html.tpl" -o misscon-report.html --security-checks config .
```

2.9 Individual Scan for vulnerabilities

```
trivy fs --format template --template "@html.tpl" -o vuln-report.html --security-checks vuln .
```

3 Scan a Docker image

3.1 Run Docker Desktop

3.2 Build the Image using CMD Ex: docker build -t trivy .

Docker image is: **trivy:latest**

3.3 full scan (vulnerabilities, misconfigurations and secrets)

```
trivy image --format template --template "@html.tpl" -o docfullscan.html --scanners vuln,secret,misconfig trivy:latest
```

3.4 Individual Scan for secret

```
trivy image --format template --template "@html.tpl" -o docsec.html --scanners secret trivy:latest
```

3.5 Individual Scan for misconfigurations

```
trivy image --format template --template "@html.tpl" -o docmiss.html --scanners misconfig trivy:latest
```

3.6 Individual Scan for vulnerabilities

```
trivy image --format template --template "@html.tpl" -o docvul.html --scanners vuln trivy:latest
```

Program 8.

Simulate and manage Amazon S3 buckets and EC2 using LocalStack locally. Perform operations using both the LocalStack Cloud Dashboard and AWS CLI.

Prerequisite Software

1. LocalStack Cloud
2. Python 3.9.6
3. LocalStack
4. Docker Desktop
5. AWS CLI

1. Simulate S3 Buckets using AWS LocalStack Cloud Dashboard

Step 1: Log into LocalstackCloud

Step 2: Select "Local Instance"

Step 3: Open the "Resource Browser"

Step 4: Click on S3 from the services menu.

Step 5: Click Create Bucket

Step 6: Enter a bucket name

Step 7: View S3 Buckets

Step 8: To upload a file to a specific S3 bucket

Step 9: view files (objects) inside an S3 bucket

Step 10: delete a bucket

2. Simulate S3 Buckets Using AWS CLI with LocalStack

2.1 AWS CLI configuration prompt: it's used to set up your AWS credentials and default settings.

```
aws configure
```

2.2 Create a bucket

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://dept-cse
```

2.3 List All Buckets

```
aws --endpoint-url=http://localhost:4566 s3 ls
```

2.4 upload file to a specific "folder"

create a text file

```
echo "Hello LocalStack" > test.txt
```

upload / copy a file to dept-cse

```
aws --endpoint-url=http://localhost:4566 s3 cp test.txt s3://dept-cse
```

2.5 Delete File and Bucket

```
aws s3 rm s3://dept-cse --recursive --endpoint-url=http://localhost:4566
```

3. Simulate EC2 Using **AWS CLI** with LocalStack

3.1 Create a Key Pair

```
aws --endpoint-url=http://localhost:4566 ec2 create-key-pair --key-name test-key
```

3.2 Run an EC2 Instance

```
aws --endpoint-url=http://localhost:4566 ec2 run-instances --image-id ami-12345678 --instance-type t2.micro --key-name test-key
```

3.3 List Running EC2 Instances

```
aws --endpoint-url=http://localhost:4566 ec2 describe-instances
```

3.4 Stop the EC2 Instance

```
aws --endpoint-url=http://localhost:4566 ec2 stop-instances --instance-ids i-283153368cc22fb2c
```

3.5 Terminate the EC2 Instance

```
aws --endpoint-url=http://localhost:4566 ec2 terminate-instances --instance-ids  
i-283153368cc22fb2c
```

3.6 Delete Key Pair

```
aws --endpoint-url=http://localhost:4566 ec2 delete-key-pair --key-name test-key
```

4. Simulate EC2 using AWS LocalStack **Cloud Dashboard**

4.1 : Open the Dashboard

4.2 : Select "Local Instance"

4.3 : Open the "Resource Browser"

4.4 : Type ec2 in the Search Bar

4.5 : Click on EC2

4.6 : Click "Launch Instance"

4.7 : fill the details of Image ID, Instance Type, Key Name and Click the Submit Button

4.8 view your EC2 instance

4.9 EC2 Instance Actions in LocalStack Cloud Dashboard

Actions 1. **Stop** Instance

2. **Start** Instance

3. **Terminate** Instance

Program 9.

Simulate and manage a **Amazon DynamoDB** Using AWS CLI with AWS LocalStack Cloud.

Prerequisite software:

1. LocalStack Cloud
2. Python 3.6.9
3. LocalStack
4. Docker
5. AWS CLI
6. configure dummy credentials using aws configure

AWS Access Key ID: test

AWS Secret Access Key: test

Default region name: us-east-1

Default output format: json

Implementation Steps

1. Create a DynamoDB Table

```
aws --endpoint-url=http://localhost:4566 dynamodb create-table ^  
--table-name Students ^  
--attribute-definitions AttributeName=USN,AttributeType=S ^  
--key-schema AttributeName=USN,KeyType=HASH ^  
--provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5
```

2. List Tables

```
aws --endpoint-url=http://localhost:4566 dynamodb list-tables
```

3. Insert Data(PutItem)

Ex:1

```
aws --endpoint-url=http://localhost:4566 dynamodb put-item ^  
--table-name Students ^  
--item "{\"USN\": {\"S\": \"1ga25cs001\"}, \"Name\": {\"S\": \"Ramesh\"}, \"Address\": {\"S\":  
\"Bangalore\"}}"
```

Ex:2

```
aws --endpoint-url=http://localhost:4566 dynamodb put-item ^  
--table-name Students ^  
--item "{\"USN\": {\"S\": \"1ga25cs002\"}, \"Name\": {\"S\": \"Naresh\"}, \"age\": {\"N\": \"40\"},  
\"Address\": {\"S\": \"Bangalore\"}}"
```

4. Retrieve Data (GetItem)

Ex1:

```
aws --endpoint-url=http://localhost:4566 dynamodb get-item ^  
    --table-name Students ^  
    --key '{"USN\": {"S\": \"1ga25cs001\"}}'
```

Ex2:

```
aws --endpoint-url=http://localhost:4566 dynamodb get-item ^  
    --table-name Students ^  
    --key '{"USN\": {"S\": \"1ga25cs002\"}}'
```

5. To Update / Rename Value (i.e., a record)

Note: you have to copy value to new key, then remove old:

Let's say you want to update the **Address** of the student with **USN = "1ga25cs001"**.

```
aws --endpoint-url=http://localhost:4566 dynamodb update-item ^  
    --table-name Students ^  
    --key '{"USN\": {"S\": \"1ga25cs001\"}}' ^  
    --update-expression "SET Address = :newaddress" ^  
    --expression-attribute-values "{\":newaddress\": {\"S\": \"Mysore\"}}"
```

Note :To check whether value has been updated or not

```
aws --endpoint-url=http://localhost:4566 dynamodb get-item ^  
    --table-name Students ^  
    --key '{"USN\": {"S\": \"1ga25cs001\"}}'
```

6. To delete an item (i.e., a record)

```
aws --endpoint-url=http://localhost:4566 dynamodb delete-item ^  
    --table-name Students ^  
    --key '{"USN\": {"S\": \"1ga25cs002\"}}'
```

Note :To check whether item has been deleted or not

Ex:

```
aws --endpoint-url=http://localhost:4566 dynamodb scan ^  
    --table-name Students ^  
    --projection-expression "USN" ^  
    --output json
```

7. To delete an attribute

EX: Address is an attribute

```
aws --endpoint-url=http://localhost:4566 dynamodb update-item ^  
  --table-name Students ^  
  --key "{\"USN\": {\"S\": \"1ga25cs001\"}}" ^  
  --update-expression "REMOVE Address"
```

Note:To check whether an attribute has been deleted or not

```
aws --endpoint-url=http://localhost:4566 dynamodb get-item ^  
  --table-name Students ^  
  --key "{\"USN\": {\"S\": \"1ga25cs001\"}}"
```

8. Delete Table

```
aws --endpoint-url=http://localhost:4566 dynamodb delete-table --table-name Students
```

Note:To check whether Table has been deleted or not

```
aws --endpoint-url=http://localhost:4566 dynamodb list-tables
```

Program 10.

Design and Develop a custom **Student Registration Form** application using Salesforce as a SaaS platform

Prerequisites

1. Salesforce Developer Account

- Sign up (if not already): <https://login.salesforce.com/>

Implementation Steps

Step 2: Create a Custom Object and Field name

2.1 Click on **Setup** →

2.2 Click on **Create** and select **Custom Object**

2.3 click on **Salesforce Classic**

2.4 Enter the details for **Custom Object**

Details:

- Label: **Student**
- Plural Label: **Students**
- Record Name : **Student Name**
- Data Type: **Text**

Click on **Save**

2.5. After **saving**, you can view the record details of the '**Student**' custom object

2.6 From the top-right corner, click the gear icon and choose 'Setup'

Step 3: Create Custom Field & Relationships on Object

3.1 "Click on 'Object Manager' to view the 'Student' custom object."

and

Click on 'Student' custom object'

3.2 Click on Field & Relationships

3.3 Click on New

3.4 click on Salesforce Classic

field name is Class

3.5 select data type as **text** for Class - **Field Name**

3.6 : Enter the details for Class

Details:

- Field Label: Class
- Length: 10

Click on **Next**

3.7 Click on **Next**

3.8 Click on **Save and New**

field name is Mobile No

3.9: select data type as **Number** for Mobile No **Field Name**

3.10 Enter the details for Mobile No

Details:

- Field Label: Mobile No
- Length: 12

Click on **Next**

3.11 click on Next

3.12. click on Save

Step 4: Create a Custom Object Tab for Student

4.1 Click on **Home - Search – tabs**

4.2 Click on **salesforce Classic**

4.3 Click on **New**

4.4 select an existing object: **Student**

4.5 **Next,Next,Save**

4.6 Select a **Tab Style**

4.7. Click on **Setup**

4.8 Go to **App Launcher** and Search and click **Students**

4.9 view the **Students tab** and click on **new** to enter new student data

Step 5: Create a Custom App using Lightning App

5.1 In the Left Sidebar, Click on “**Home**”

5.2 Search and click **App Manager**.

5.3 Click **New Lightning App**.

5.4 App Name: **Student Registration Form** and **Upload an Image**, click **Next**.

5.5 Choose **Standard Navigation** and Desktop, click **Next**.

5.6 Add utility items if needed (skip for now), click **Next**.

5.7 Add Custom Tab (EX: Student) , click **Next**.

Select **the student tab** from Available Items. Here we can add multiple tab.

Example:

5.8 Add **User Profiles** (select `System Administrator`), click **Save & Finish**.

5.9 View of a Lightning App

Step 6: Test the App / Launch the App

6.1 Go to **App Launcher (Grid Icon)**

6.2 Click on View to Displays All Apps

6.3 Click on Student Registration Form

6.4 Click on **Students** → click on new to enter new student data

6.5 Enter Student Details and save it

6.6 Click on the Record to view the Details

6.7 view the Details